TOKENTO

Programmable Merchant Value Token Network

Internal Product Specification

Version 1.0 | Confidential

Classification: Internal Only

Not for external distribution without explicit written approval.

I. Executive Summary

Tokento is a programmable merchant value token network built for the emerging agentic commerce economy. Merchants issue structured value tokens — covering loyalty rewards, dynamic pricing incentives, returns credits, and trial offers — through a single API. Those tokens travel with the customer in a platform-managed wallet, are queryable by any AI agent regardless of which protocol it operates on, and are redeemable at checkout automatically.

Today's loyalty infrastructure was designed for a human-initiated world: customers log into portals, remember points balances, and manually apply rewards. This model breaks entirely in agentic commerce, where AI agents execute purchases on behalf of users with no interaction surface for traditional loyalty mechanics. Tokento fixes this at the infrastructure layer — making merchant incentives machine-readable, portable, and agent-executable for the first time.

Dimension	Summary
Company name	Tokento

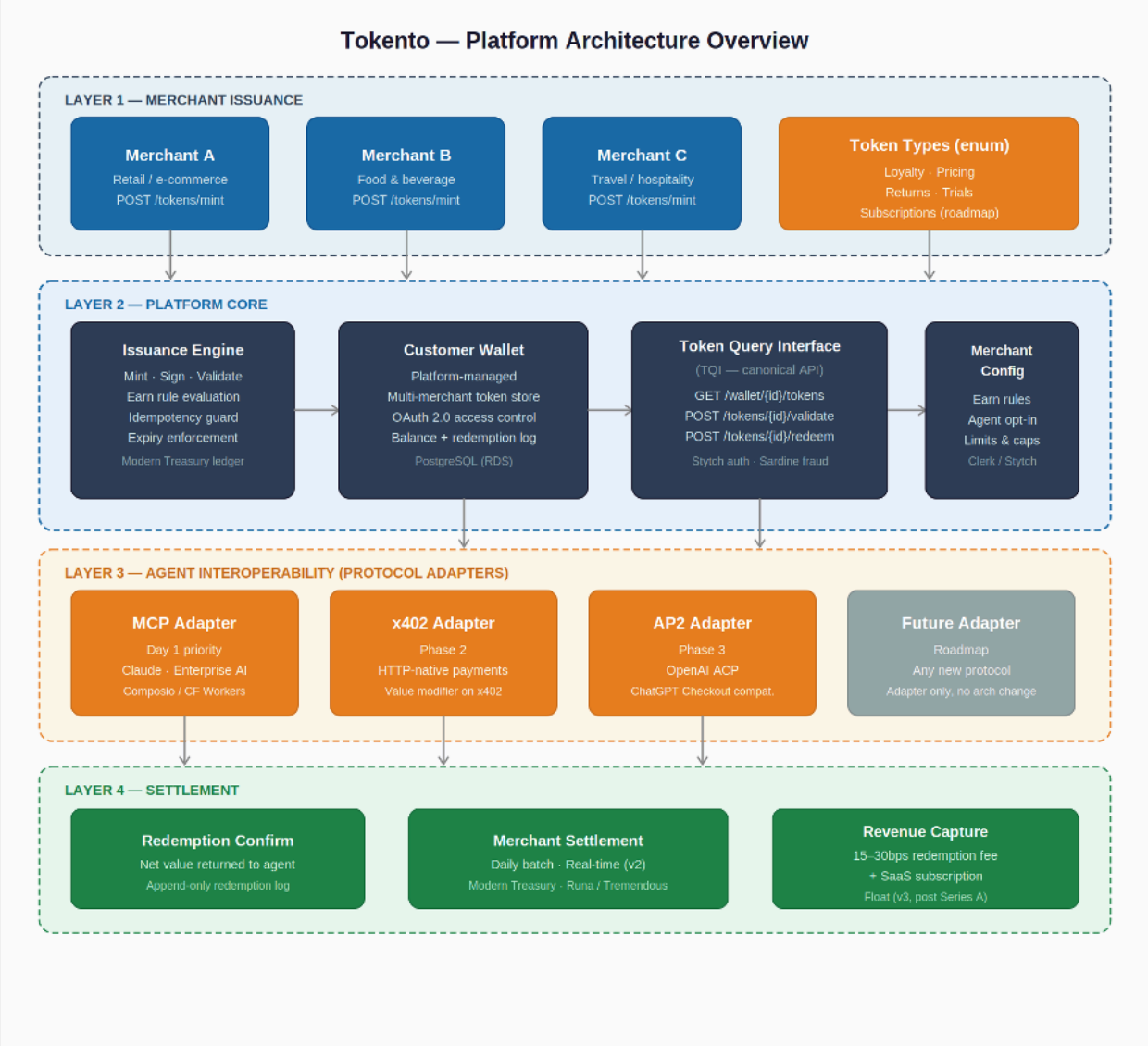
Product type	B2B SaaS — API-first token issuance and wallet platform
Primary customer	Merchants seeking to make loyalty and incentive programs agent-accessible
Core technology	Token issuance engine, platform-managed customer wallet, Token Query Interface (TQI)
Agent compatibility	Protocol-agnostic via adapter layer: MCP, x402, AP2, and future protocols
Revenue model	Per-redemption interchange fee (15–30bps) + monthly SaaS subscription
Token types (v1)	Loyalty — with Pricing, Returns, and Trials on the expansion roadmap
MVP timeline	3–4 weeks with recommended build stack (vs. 6 weeks full custom)

Tokento is not a loyalty company with a technology angle. It is infrastructure — the value layer that sits beneath every agentic transaction and ensures merchant incentives survive the transition to machine-driven commerce.

Platform Architecture Overview

The diagram below illustrates Tokento's four-layer architecture — from merchant issuance at the top, through the platform core (issuance engine, customer wallet, TQI), the agent interoperability adapters, and down to the settlement layer.

Figure 1 — Tokento Platform Architecture: four-layer model from merchant issuance to agent-executed settlement



II. The Problem

2.1 Loyalty infrastructure was built for humans

Every component of today's loyalty stack assumes a human actor in the loop. Redemption portals require login. Punch cards require physical presence. Points balances require the customer to remember, check, and choose. The entire model is predicated on conscious human-initiated action at the moment of purchase.

That model is structurally incompatible with agentic commerce. AI agents acting as purchasing proxies have no surface area to interact with loyalty portals, no mechanism to retrieve a customer's fragmented points balances, and no protocol for presenting loyalty credentials at

checkout. Merchant incentives become functionally invisible the moment an agent enters the transaction.

2.2 The agentic commerce transition is accelerating

OpenAI's Agentic Commerce Protocol, Anthropic's MCP ecosystem, and payment primitives like x402 are collectively moving commerce toward machine-to-machine transaction execution. The infrastructure layer that handles payments (Stripe), identity (Okta), and bank connectivity (Plaid) already exists for agentic commerce. The value layer — encoding and delivering merchant incentives within machine-executed transactions — does not. That gap is Tokento's market.

2.3 The loyalty market is large and structurally broken

The global customer loyalty market represents over $5 trillion in issued value annually. Existing SaaS loyalty platforms — Talon.One, Antavo, Yotpo, LoyaltyLion, Annex Cloud — address human-initiated redemption with increasing sophistication. None of them address agent-initiated redemption. Their architectures are UI-first: dashboards, customer portals, mobile apps. There is no Token Query Interface, no machine-readable wallet, no agent-presentable credential format.

2.4 The fragmentation problem compounds the gap

A customer's loyalty value is distributed across dozens of merchant-siloed programs with incompatible formats and no shared addressing scheme. An AI agent optimizing a purchase has no practical mechanism to discover, aggregate, or apply that value. Tokento solves fragmentation at the issuance layer: by being the system of record through which merchants mint tokens, Tokento becomes the natural aggregation point — a single wallet any authorized agent can query.

III. Vision

3.1 The vision statement

Loyalty programs were built for a world where humans shop. In the emerging agentic economy, AI agents act as purchasing proxies — researching options, comparing prices, negotiating terms, and completing transactions on behalf of customers with minimal human intervention. These agents are fast, rational, and relentless optimizers. But they cannot log into a rewards portal. They cannot apply a punch card. They cannot remember that a customer has value sitting in a merchant loyalty account.

Existing loyalty infrastructure becomes functionally invisible the moment an agent enters the transaction — not because loyalty stops mattering, but because the infrastructure that delivers it was never designed for machines.

Tokento is the programmable merchant value token network for the agent economy. Merchants issue structured value tokens through a single API. Tokens travel with the customer, are queryable by any AI agent, and are redeemable at checkout across any protocol. For the first time, merchant incentives become a first-class citizen of the agentic transaction stack.

3.2 Token type expansion roadmap

Token type	Description	Availability
Loyalty	Earn-and-redeem rewards issued on qualifying transactions	v1 — MVP
Pricing	Dynamic discount tokens encoding conditional price reductions	v2 — Post-MVP
Returns	Store credit tokens issued post-return, applicable to future purchases	v2 — Post-MVP
Trials	First-purchase incentive tokens for new customer acquisition	v3 — Roadmap
Subscriptions	Recurring value tokens tied to membership tiers	v3 — Roadmap

3.3 Infrastructure positioning

Stripe abstracted payments. Plaid abstracted bank connectivity. Okta abstracted identity. Tokento abstracts merchant value — making it portable, programmable, and universally agent-accessible. This is infrastructure, not an application. It earns value through ubiquity and through sitting beneath every agentic transaction that involves merchant incentives.

IV. Token Architecture

4.1 What a token is

A Tokento token is a structured, merchant-issued value object with a defined lifecycle: Mint → Hold → Present → Validate → Redeem → Settle. It is not a blockchain token. It lives on a traditional high-performance ledger with token semantics as the API abstraction — prioritizing speed, regulatory simplicity, and merchant familiarity.

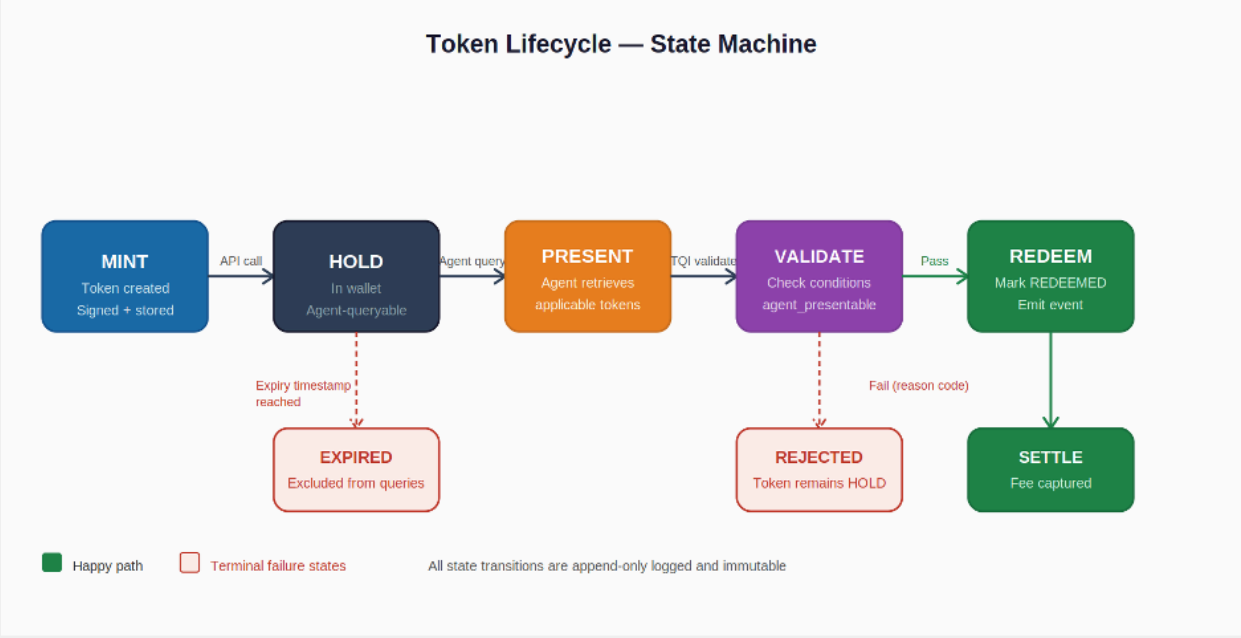
4.2 Token object schema

Field group	Fields and description
Identity	token_id (UUID, globally unique) · issuing_merchant_id · customer_wallet_id · issue_timestamp · expiry_timestamp · token_type enum: loyalty | pricing | returns | trial
Value	denomination (USD or merchant-defined unit) · partial_redemption_allowed (boolean) · minimum_transaction_floor (USD)
Conditions	category_restriction (SKU-level, category-level, or unrestricted) · channel_restriction (online | agent-initiated | in-store | all) · stackability_flag (boolean) · agent_presentable_flag (explicit merchant opt-in)
Settlement	settlement_type (discount | cashback | credit) · settlement_timing (real-time | daily_batch) · merchant_liability_account_ref
Audit	created_by · last_modified · redemption_log (append-only, immutable) · schema_version

4.3 Token lifecycle state machine

The diagram below illustrates the six-stage token lifecycle including the two terminal failure states (REJECTED and EXPIRED) and their trigger conditions.

Figure 2 — Token Lifecycle State Machine: six stages from Mint to Settle, with REJECTED and EXPIRED terminal states



4.4 Merchant configuration surface

- Earn rules: spend threshold → token denomination mapping (e.g., $100 spend → $5 loyalty token)
- Token parameters: expiry window, category restrictions, channel restrictions, stackability flag
- Redemption caps: maximum redemption value per customer per period, maximum tokens per transaction
- Agent opt-in toggle: explicit per-merchant enable/disable for agent-initiated redemption (off by default)
- Settlement preference: real-time vs. daily batch; cashback vs. discount vs. credit

V. Agent Interoperability Layer

5.1 Design philosophy: protocol-agnostic by architecture

Tokento is protocol-agnostic. The platform does not pick winners among competing agent protocols — it serves all of them through a canonical internal API (TQI) with thin protocol-specific adapters layered on top. Adding compatibility with a new agent protocol is an adapter problem, not an architecture problem.

5.2 The Token Query Interface (TQI)

Endpoint	Method + path	Function
Query wallet	GET /wallet/{customer_id}/tokens	Returns all ACTIVE, non-expired tokens. Filterable by merchant_id, category, channel, token_type, min_denomination.
Validate token	POST /tokens/{token_id}/validate	Checks presentability for a given transaction context. Returns detailed failure reason on rejection.
Redeem token	POST /tokens/{token_id}/redeem	Executes redemption. Idempotent. Returns confirmed net transaction value. Initiates settlement cycle.

5.3 Protocol adapter prioritization

Protocol	Implementation detail and priority
MCP — Day 1	TQI exposed as MCP server with three named tools: query_loyalty_tokens, validate_token, redeem_token. Any MCP-compatible agent can call Tokento natively. Hosted via Composio + Cloudflare Workers.
x402 — Phase 2	Tokento intercepts x402 payment requests, queries applicable tokens, modifies the payment amount, and passes the net to x402 for settlement. Enhances x402, does not replace it.
AP2 / OpenAI ACP — Phase 3	OpenAI's checkout protocol. Adapter built but not led with in early GTM. MCP story is broader and cleaner for the first 12 months.
Future protocols — Ongoing	New adapter only. TQI remains the stable internal contract regardless of protocol landscape evolution.

5.4 End-to-end agent transaction flow

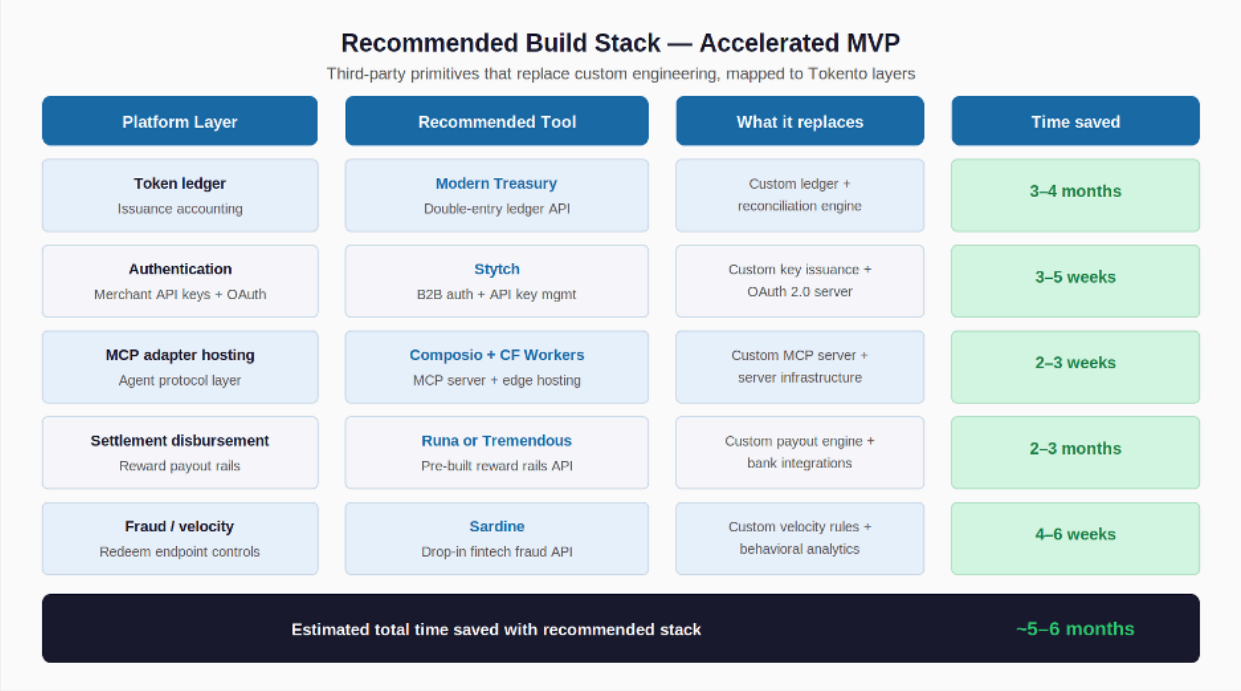
- Customer grants Tokento wallet access to their AI agent during onboarding (OAuth 2.0).
- Agent receives a purchase task and identifies a candidate merchant.
- Pre-checkout: agent calls TQI GET /wallet/{customer_id}/tokens with merchant_id and channel=agent-initiated filter.
- TQI returns applicable tokens (e.g., $25 loyalty token, not expired, agent_presentable_flag = true).
- Agent calls TQI POST /tokens/{token_id}/validate with transaction amount and context.
- Validation passes. Agent includes token reference in checkout payload alongside payment credential.
- Tokento processes redemption, returns net transaction value to agent, initiates merchant settlement.
- Agent completes purchase at net price. Customer sees loyalty applied without any manual action.

VI. Recommended Build Stack

Tokento's core differentiation lives in the token issuance engine, the TQI logic, and the earn rules configuration surface. Everything else — ledger accounting, authentication, MCP hosting,

settlement rails, fraud controls — can be composed from best-in-class third-party primitives. The recommended stack below compresses the MVP build timeline from approximately 6 weeks of full custom engineering to 3–4 weeks.

Figure 3 — Recommended Build Stack: third-party primitives mapped to Tokento layers with estimated time saved



6.1 Stack detail and rationale

Tool	Rationale and integration point
Modern Treasury (ledger)	Purpose-built double-entry ledger API. Handles token issuance and redemption accounting without building reconciliation from scratch. API-first, well-documented. Saves 3–4 months of custom ledger engineering. Integration point: Issuance Engine calls Modern Treasury on every mint and redeem event.
Tigerbeetle (alternative)	Open-source, extremely high-performance financial ledger. Best option if you want full ledger ownership with no per-transaction SaaS fee. Steeper integration effort. Recommended at Series A scale when transaction volume makes Modern Treasury pricing material.
Stytch (auth)	B2B-native auth platform with built-in API key management and OAuth 2.0 server. Handles both merchant API keys and customer wallet access tokens in one platform. Saves 3–5 weeks. Integration point: all TQI endpoints validate through Stytch.

Composio + Cloudflare Workers (MCP hosting)	Composio manages MCP server registration and tool discovery. Cloudflare Workers provides edge hosting for the MCP adapter service with sub-50ms global latency. Saves 2–3 weeks of MCP infrastructure setup. Integration point: MCP adapter layer translates tool calls into TQI HTTP requests.
Runa or Tremendous (settlement)	Pre-built reward disbursement rails with API-first interfaces. Runa specializes in digital rewards (gift cards, prepaid, vouchers). Tremendous is broader (ACH, PayPal, checks). Either eliminates the need to build custom payout logic and bank integrations. Saves 2–3 months. Integration point: Settlement Service calls Runa/Tremendous after redemption confirmation.
Sardine (fraud)	Purpose-built fintech fraud platform. Drop-in risk scoring API with pre-built velocity rules, behavioral analytics, and device intelligence. Plugs into the redeem endpoint as a pre-execution check. Saves 4–6 weeks of custom fraud engineering. Integration point: Redemption Service calls Sardine risk score before processing each redeem request.
Alviere / Bond (future — v3)	White-label stored-value and wallet infrastructure. Relevant when the customer wallet evolves to hold actual monetary value (float revenue stream). Not needed in MVP where the wallet is a token store, not a monetary account.
Highnote / Marqeta (future — v3)	Card issuing infrastructure. Relevant if Tokento eventually issues a co-branded card that automatically applies loyalty tokens at POS. Out of scope for MVP and v2.

VII. Revenue Model

Revenue stream	Detail
Primary: Interchange fee (15–30bps)	Assessed on redeemed token face value at redemption. Aligns platform revenue with merchant value delivery. Agentic redemptions are structurally higher-intent than traditional loyalty, expanding the revenue base over time.
Secondary: SaaS subscription ($199–$499/mo)	Per-merchant monthly fee for API access, configuration dashboard, analytics, and support SLA. Provides predictable base ARR. Acts as commitment signal filtering low-intent merchants.
Tertiary (v3): Float on unredeemed liability	Loyalty breakage averages 20–30% of issued value. At scale, Tokento holds this float and earns yield. Requires stored-value regulatory

	compliance. Build toward post-Series A with appropriate counsel. Do not pursue in MVP.

7.1 Unit economics illustration

Metric	Conservative (Mo. 12)	Base case (Mo. 12)
Merchants on platform	50	150
Avg SaaS subscription / merchant / month	$299	$349
Monthly redemption volume / merchant	$50,000	$80,000
Platform interchange fee	20bps	20bps
Monthly interchange revenue / merchant	$100	$160
Total monthly revenue	$19,950	$76,350
Annualized run rate	$239,400	$916,200

VIII. MVP Scope

8.1 MVP principle

The MVP validates one core hypothesis: merchants will pay to make their loyalty programs agent-accessible, and AI agents will successfully query and apply loyalty tokens during agentic checkout. Every MVP capability is selected to validate or de-risk this hypothesis within a 3–4 week build window using the recommended stack.

MVP is scoped to loyalty token type only. Pricing, returns, and trial token types are post-MVP. MCP is the only agent protocol adapter in MVP. x402 and AP2 follow in Phase 2.

8.2.1 Merchant onboarding (MO)

ID	Capability / Feature	Description	Priority	Success Criteria
MO-01	**API key issuance**	Merchant registers via dashboard and receives a scoped API key for token issuance and configuration. Keys are merchant-scoped with rate limits.	**P0**	Merchant can authenticate and issue a test token within 10 minutes of registration.
MO-02	**Earn rule configuration**	Merchant defines earn rules: spend threshold → token denomination. Multiple rules per merchant supported.	**P0**	Earn rules persist, are retrievable via API, and correctly trigger token minting at the configured threshold.
MO-03	**Token parameter config**	Merchant sets expiry window (days), category restriction, channel restriction, and stackability flag per earn rule.	**P0**	Tokens minted under a rule inherit configured parameters. Parameters are validated server-side at configuration time.
MO-04	**Agent opt-in toggle**	Merchant explicitly enables agent_presentable_flag at account level and per earn rule. Off by default.	**P0**	Tokens with agent_presentable_flag = false are rejected by TQI validate. Toggle change takes effect within 60 seconds.
MO-05	**Settlement preference**	Merchant selects settlement type (discount | cashback | credit) and timing (real-time | daily_batch) at onboarding.	**P1**	Settlement preference stored and applied correctly during post-MVP redemption settlement cycle.
MO-06	**Merchant dashboard**	Web dashboard showing issued tokens, redemption activity, customer wallet stats, and revenue impact. Read-only analytics in MVP.	**P1**	Dashboard loads within 2 seconds. Data reflects redemption events within 5-minute lag.

8.2.2 Token issuance engine (TI)

ID	Capability / Feature	Description	Priority	Success Criteria
TI-01	**Mint endpoint**	POST /tokens/mint — accepts merchant_id, customer_id, transaction_amount, earn_rule_id. Returns signed token object if threshold met.	**P0**	Token minted and returned within 200ms p99. Token immediately queryable in customer wallet.
TI-02	**Token signing**	All tokens signed at mint time using HMAC-SHA256 with merchant-scoped key. Prevents token forgery and tampering.	**P0**	Tampered tokens rejected at validation with 401. Signing adds less than 5ms to mint latency.
TI-03	**Expiry enforcement**	Tokens with expiry_timestamp in the past automatically excluded from TQI wallet query results and rejected at validate.	**P0**	Expired tokens never appear in agent query results. Expiry evaluated at request time, not batch.
TI-04	**Duplicate mint guard**	Idempotency key support on mint endpoint. Same transaction_id + merchant_id returns existing token rather than minting duplicate.	**P0**	Duplicate mint attempt returns original token with HTTP 200 (not 201). No duplicate tokens in wallet.
TI-05	**Partial redemption**	If partial_redemption_allowed = true, agent can redeem a fraction of token face value. Remaining balance persists in wallet.	**P1**	Partial redemption correctly reduces token balance. Cannot exceed face value.
TI-06	**Stackability enforcement**	If stackability_flag = false, TQI validate rejects a second token from the same merchant in the same transaction.	**P1**	Non-stackable tokens correctly blocked at validate. Error message specifies conflicting token_id.

8.2.3 Customer wallet (CW)

ID	Capability / Feature	Description	Priority	Success Criteria
CW-01	**Wallet creation**	Platform-managed wallet created automatically on first token mint for a customer_id. No customer registration step required in MVP.	**P0**	Wallet created within first mint response. customer_id is merchant-provided external reference.

CW-02	**Wallet query (TQI GET)**	GET /wallet/{customer_id}/tokens — returns all ACTIVE, non-expired tokens. Supports filters: merchant_id, category, channel, token_type, min_denomination.	**Po**	Query returns within 100ms p99. Empty wallet returns []. Filters correctly exclude non-matching tokens.
CW-03	**Multi-merchant wallet**	Single customer wallet holds tokens from multiple merchants. Tokens are merchant-scoped and do not bleed across merchant contexts.	**Po**	Tokens from Merchant A are not returned in a query filtered to Merchant B. Cross-merchant contamination = 0.
CW-04	**Wallet authorization**	Agents must present a customer-granted wallet access token (OAuth 2.0 bearer) to query or redeem. Merchants cannot query other merchants' customers.	**Po**	Unauthenticated requests return 401. Cross-merchant queries return 403. Auth model passes OWASP top-10 review.
CW-05	**Redemption log**	Append-only log of all redemption events per token. Accessible to merchant via dashboard and API. Immutable after write.	**Po**	Every redemption event logged within 100ms. Log cannot be modified via any API endpoint. Merchant can export CSV.

8.2.4 Token Query Interface — validate and redeem (TQ)

ID	Capability / Feature	Description	Priority	Success Criteria
TQ-01	**Validate endpoint**	POST /tokens/{id}/validate — checks: not expired, merchant match, amount >= floor, agent_presentable_flag, channel match, not already redeemed.	**Po**	Validation completes within 80ms p99. Returns detailed failure reason (not generic error) on rejection.
TQ-02	**Redeem endpoint**	POST /tokens/{id}/redeem — executes redemption after implicit re-validation. Returns net_transaction_value and settlement_reference. Marks token REDEEMED.	**Po**	Redemption is idempotent: second call on same token returns original result, does not double-redeem.

TQ-03	**MCP tool exposure**	TQI endpoints exposed as three named MCP tools: query_loyalty_tokens, validate_token, redeem_token. Hosted via Composio + Cloudflare Workers.	**P0**	MCP tools discoverable via standard MCP tool listing. Claude can invoke all three tools without custom client code.
TQ-04	**Rate limiting**	Per-merchant and per-agent rate limits on all TQI endpoints. Default: 100 req/min query, 20 req/min validate, 10 req/min redeem.	**P0**	Requests exceeding limits return 429 with Retry-After header. Limits configurable per merchant in admin console.
TQ-05	**Fraud velocity controls**	Via Sardine integration: max redemptions per customer_id per hour, max redemption value per day, anomaly flagging on sudden balance changes.	**P1**	Velocity thresholds configurable per merchant. Flagged events surface in merchant dashboard within 2 minutes.

8.2.5 Platform and trust (PT)

ID	Capability / Feature	Description	Priority	Success Criteria
PT-01	**Audit logging**	All API calls (mint, query, validate, redeem) logged with timestamp, caller identity, parameters, and response code. Retained 90 days minimum.	**P0**	100% of API calls in audit log. Log queryable by merchant_id, customer_id, token_id, and date range.
PT-02	**Data encryption**	All data encrypted at rest (AES-256) and in transit (TLS 1.3 minimum). Keys managed via AWS KMS.	**P0**	TLS 1.2 and below connections rejected. At-rest encryption verified via cloud provider compliance report.
PT-03	**API versioning**	All TQI endpoints versioned under /v1/. Breaking changes deployed under /v2/ with 6-month deprecation window.	**P0**	v1 endpoints return API-Version header. Version mismatch returns 400 with upgrade guidance.
PT-04	**Sandbox environment**	Full sandbox mirroring production. Sandbox tokens never settle real value. Merchant onboarding begins in sandbox before production access.	**P0**	Sandbox available at 99.9% uptime. Sandbox tokens rejected by production redeem with clear error.

PT-05	**Webhook notifications**	Merchants receive webhook events for: token_minted, token_redeemed, token_expired, velocity_alert. Configurable endpoint per merchant.	P1	Webhook delivery within 5 seconds of event. Failed deliveries retried 3x with exponential backoff.

IX. Technical Architecture

9.1 Architecture principles

- API-first: every product capability accessible via API before any dashboard UI is built
- Stateless services: all application logic is stateless; state lives in the datastore and cache layers
- Event-driven settlement: redemption events emit to a settlement queue; workers process asynchronously
- Adapter pattern for protocols: TQI is the stable internal contract; adapters translate external protocols
- Zero-trust authorization: every API call authenticated and authorized; no implicit trust between services

9.2 Service decomposition

Service	Responsibility
Token Issuance Service	Handles mint endpoint. Evaluates earn rules, generates token objects, signs via HMAC-SHA256, writes to token store. Emits token_minted events to EventBridge.
Wallet Service	Maintains customer wallet state. Handles GET /wallet queries with filter logic. Manages ACTIVE / REDEEMED / EXPIRED state transitions.
Validation Service	Stateless validation logic. Reads token from Wallet Service, evaluates all predicates, returns pass/fail with detailed reason code.
Redemption Service	Handles POST /redeem. Re-validates, calls Sardine risk score, writes REDEEMED state (idempotent), emits token_redeemed event, returns net_transaction_value.

Settlement Service	Consumes token_redeemed events. Calculates merchant liability delta. Calls Runa/Tremendous for disbursement. Calls Modern Treasury for ledger update. Captures platform fee.
MCP Adapter Service	Translates MCP tool calls into TQI HTTP calls. Hosted on Cloudflare Workers. Registered in Composio MCP directory.
Merchant Config Service	Stores and serves merchant configuration: earn rules, token parameters, settlement preferences, agent opt-in flags.
Auth Service	Stytch integration. Issues and validates merchant API keys and customer OAuth 2.0 wallet access tokens.
Audit Service	Receives log events from all services via EventBridge. Writes to immutable audit log store. Serves log query API to merchants.
Notification Service	Consumes events and dispatches webhooks to merchant endpoints. Manages retry logic and delivery log.

9.3 Infrastructure stack

Component	Technology choice (MVP)
Cloud provider	AWS (primary)
API gateway	AWS API Gateway with Stytch custom authorizer
Application services	Node.js or Go microservices, Docker containers, ECS Fargate
Primary datastore	PostgreSQL (RDS) — tokens, wallets, merchants, earn rules
Cache layer	Redis (ElastiCache) — wallet query results, rate limit counters, earn rule lookups
Event bus	Amazon EventBridge — decouples issuance, redemption, settlement, notification services
Ledger	Modern Treasury — token issuance and redemption accounting
Auth	Stytch — merchant API keys + customer OAuth 2.0 wallet tokens
MCP hosting	Composio + Cloudflare Workers — edge-hosted MCP adapter
Settlement rails	Runa or Tremendous — reward disbursement API
Fraud	Sardine — risk scoring on redeem endpoint
Secret management	AWS KMS for HMAC signing keys; Secrets Manager for service credentials
Monitoring	Datadog APM + CloudWatch alarms

CI/CD	GitHub Actions → ECR → ECS rolling deploys

X. Go-to-Market

10.1 Beachhead strategy

Tokento's initial GTM targets mid-market e-commerce and retail merchants who (a) already have a loyalty program, (b) are aware of agentic commerce as an emerging channel, and (c) have sufficient transaction volume to generate meaningful redemption economics. These merchants feel the loyalty invisibility problem acutely but lack engineering resources to solve it themselves.

10.2 Phase 1: Pilot program (Weeks 1–12 post-MVP)

- Target 5–10 design partner merchants at no cost in exchange for usage data, testimonials, and product feedback.
- Focus on one merchant category for the first cohort to build a vertical reference customer.
- Instrument every interaction: which agents call TQI, redemption rates, token abandonment, dashboard engagement.
- Use pilot data to validate unit economics assumptions and prioritize the highest-value token types for Phase 2.

10.3 Phase 2: Paid launch (Months 3–9)

- Convert design partners to paid SaaS subscriptions at introductory pricing ($199/month).
- Launch x402 adapter — expands addressable agent ecosystem beyond MCP-only.
- Open inbound waitlist for merchant self-serve onboarding.
- Publish API documentation, developer guides, and sample MCP tool integrations on public GitHub.
- Target 50 paid merchants by end of Month 9.

10.4 Phase 3: Scale and token expansion (Months 9–18)

- Launch Pricing token type — dynamic discount tokens, expanding use cases and redemption volume.
- Launch Returns token type — post-return credit tokens, adding returns-heavy retail as a new category.
- Launch AP2 adapter — adds OpenAI agent ecosystem compatibility.

- Explore coalition loyalty: tokens earned at Merchant A, redeemable at Merchant B within Tokento network.
- Target 150+ merchants and Series A fundraise with 12 months of paid revenue data.

10.5 Competitive positioning

Competitor type	Tokento's differentiation
Traditional loyalty SaaS (Talon.One, Antavo, LoyaltyLion)	UI-first, human-redemption-first. No agent interoperability layer, no TQI, no MCP compatibility. Complementary short-term, replacement long-term as agentic commerce scales.
Payment infrastructure (Stripe, Adyen)	Handle money movement, not value encoding. No concept of a merchant-issued loyalty token, earn rule engine, or agent-queryable wallet.
Agent platform native loyalty (OpenAI ChatGPT Checkout)	Closed ecosystems serving one agent provider's merchant network. Tokento is protocol-agnostic — serves all agent ecosystems simultaneously. Structural advantage as the landscape fragments.
Large merchant DIY	Enterprise merchants with significant engineering could build in-house. Tokento's advantage: time-to-market, cross-merchant wallet aggregation, and protocol adapter maintenance as standards evolve.

— End of Document —